

Context Window Experiment — Comparison of Broad vs Narrow Scope Responses

Two prompts were tested to evaluate how Claude Code performs under different context scopes.

The **first prompt** intentionally required reading many interconnected modules across the axum framework:

Explain the complete lifecycle of an incoming HTTP request in axum, starting from Router::call() until the final HTTP response is produced. Include how route matching works, how URL parameters are inserted into Request::extensions, how extractors retrieve data, how MethodRouter dispatches by HTTP method, and how responses are converted into HTTP responses. Include relevant file paths and explain the interaction between the routing, extract, middleware, and response modules.

This response required Claude to read and reason across many files simultaneously, including:

- routing/mod.rs
- path_router.rs
- method_routing.rs
- url_params.rs
- extract/path/mod.rs
- extract/state.rs
- json.rs
- response/mod.rs
- handler/mod.rs
- serve/mod.rs

The resulting answer was extremely comprehensive and architecturally rich. It correctly described:

- request entry through Router::call(),
- radix-tree route matching using matchit,
- insertion of URL parameters into Request::extensions,
- extractor execution order,
- rejection short-circuiting,
- and final response conversion through IntoResponse.

It also contained detailed reasoning about middleware flow and Tower service composition.

However, the broad context produced several weaknesses:

- the response became extremely long,
- some explanations drifted into implementation details unrelated to the original question,
- architectural layers occasionally blurred together,
- and certain sections mixed routing, extraction, and middleware responsibilities in ways that were harder to verify manually.

The **second experiment** intentionally narrowed the scope:

Focus only on the request flow involving:

- axum/src/routing/mod.rs
- axum/src/routing/path_router.rs
- axum/src/routing/method_routing.rs

The narrower response was shorter, more focused, and significantly easier to audit manually. It concentrated specifically on:

- Router::call_with_state,
- PathRouter::call_with_state,
- radix-tree matching,
- RouteId lookup,
- insertion of UrlParams,
- and MethodRouter dispatch logic.

Compared to the broader response:

- file references were more precise,
- explanations stayed tightly aligned with the prompt,
- there were fewer speculative architectural statements,
- and the reasoning chain was easier to validate against source code.

The narrower scope therefore produced a more reliable technical explanation, even though it was less comprehensive overall.

What I Learned About Context Management

This experiment demonstrated that Claude Code performs best when:

- the scope is constrained,
- the relevant files are explicitly listed,
- and unrelated architectural layers are excluded from the prompt.

Broad prompts are useful for:

- onboarding,
- architectural exploration,
- and generating high-level mental models.

However, they increase the probability of:

- abstraction drift,
- speculative reasoning,
- and subtle hallucinations caused by context overload.

Narrow prompts produce:

- more precise answers,
- better file-level grounding,
- stronger consistency,
- and lower hallucination rates.

The experiment also showed that `/clear` is important for resetting context between large investigations. Without clearing previous conversational state, Claude sometimes reused earlier assumptions and mixed unrelated implementation details into later answers.

Overall, effective **context management in Claude Code** requires:

1. limiting the active working set,
2. explicitly naming files/modules,
3. constraining output expectations,
4. and separating exploration prompts from auditing prompts.

This reinforced an important lesson: smaller, well-scoped prompts are significantly more trustworthy for deep technical analysis than large open-ended requests.